

15 | tokio编程：在多任务之间操作同一片数据

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们一起学习如何在 tokio 的多个任务之间共享同一片数据。

并发任务之间如何共享数据是一个非常重要的课题，在所有语言中都会碰到。不同的语言提供的方案支持不尽相同，比如 Erlang 语言默认只提供消息模型，Golang 也推荐使用 channel 来在并发任务之间进行同步。

Rust 语言考虑到其应用领域的广泛性和多样性，提供了多种机制来达到这一目的，需要我们根据不同的场景自行选择最合适的机制。所以相对来说，Rust 在这方面要学的知识点要多一些，好处是它在几乎所有场景中都能做到最好。

任务目标

定义一个内存数据库 db，在不同的子任务中，并发地向这个内存数据库更新数据。

潜在问题

为了简化问题，我们把 `Vec<u32>` 当作 db。比如这个 db 中现在有 10 个数据。

[复制代码](#)

```
1 let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
```

现在有两个任务 `task_a` 和 `task_b`，它们都想更新 db 里的第 5 个元素的数据 `db[4]`。

`task_a` 想把它更新成 50，`task_b` 想把它更新成 100。这两个任务之间是没有协同机制的，也就是互相不知道对方的存在，更不知道对方要干嘛。于是就可能出现这样的情况，两个任务几乎同时发起更新请求，假如 `task_a` 领先一点点时间，先把 `db[4]` 更新成 50 了，但是它得校验一下更新正确了没有，所以它得发起一个索引请求，把 `db[4]` 的数据取出来看看是不是 50。

但是在 `task_a` 去检查 `db[4]` 的值之前极小的一个时间片段里面，`task_b` 对 `db[4]` 更新操作也发生了，于是 `db[4]` 被更新成了 100。然后 `task_a` 取回值之后，发现值是 100。很奇怪，并且判断自己没有更新成功这个数据，有可能会再更新一次，再次把 `db[4]` 置为 50。这又可能对 `task_b` 的校验机制造成干扰。于是整个系统就开始紊乱了。

这就是多个任务操作共享数据可能会发生的问题。下面我们看看在 Rust 中怎么解决这个问题。

方案尝试

我们可以先从最简单的思路开始，设计如下方案。

方案一：全局变量

如果你有其他语言编程经验的话，应该很容易就能想到一个方案，就是利用全局变量来实现多个任务的数据共享。假如我们有一个全局变量 DB，为 `Vec<u32>` 类型，每个任务都可以访问到这个 DB，并可以向里面 push 数据。

我们先来试试直接在 `main` 函数里对这个 DB 全局变量进行操作。

 复制代码

```

1 static DB: Vec<u32> = Vec::new();
2
3 fn main() {
4     DB.push(10);
5 }

```

发现不行，编译会报错：

 复制代码

```

1 error[E0596]: cannot borrow immutable static item `DB` as mutable
2 --> src/main.rs:4:5
3   |
4 4 |     DB.push(10);
5   |     ^^^^^^^^^^^^^ cannot borrow as mutable

```

可能是没加 mut? 加上试一试。

 复制代码

```

1 static mut DB: Vec<u32> = Vec::new();
2
3 fn main() {
4     DB.push(10);
5 }

```

还是出错：

 复制代码

```

1 error[E0133]: use of mutable static is unsafe and requires unsafe function or block
2 --> src/main.rs:4:5
3   |
4 4 |     DB.push(10);
5   |     ^^^^^^^^^^^^^ use of mutable static
6   |
7   = note: mutable statics can be mutated by multiple threads: aliasing violations

```

Rust 编译器报错说，要使用可变的静态全局变量是不安全的，需要用到 unsafe 功能。

unsafe 功能我们还没讲，而且不是这节课的重点，所以这里不展开。总之，**Rust 不推荐我们使用全局（静态）变量**。因为全局变量不是一个好的方案，特别是对于多任务并发程序来说，应该尽可能避免。

那既然全局变量不能用了，有一个可行的选择是，在 main 函数中创建一个对象实例，把这个实例传给各个任务。因为这个实例是在 main 函数中创建的，它的生命期跟 main 函数一样长，所以也相当于全局变量了。

方案二：main 函数中的对象

稍微改一下上面的代码，这下可以了。

 复制代码

```
1 fn main() {
2     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
3     db[4] = 50;
4 }
```

下面我们尝试在 main 函数中创建一个 tokio 任务。

 复制代码

```
1 #[tokio::main]
2 async fn main() {
3     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
4
5     let task_a = tokio::task::spawn(async {
6         db[4] = 50;
7     });
8     _ = task_a.await.unwrap();
9
10    println!("{:?}", db);
11 }
```

这是一段稀松平常的代码，目的就是起一个 task，更新一下 Vec 里的元素，然后等待这个任务结束，打印这个 Vec 的值。但是，在 Rust 中，这段代码无法通过，Rust 编译器会报错。

```

1 error[E0373]: async block may outlive the current function, but it borrows `db`,
2 --> src/main.rs:5:37
3 |
4 5 |         let task_a = tokio::task::spawn(async {
5 |             -----^
6 6 |             db[4] = 50;
7 |             -- `db` is borrowed here
8 7 |         });
9 |         |_____^ may outlive borrowed value `db`
10 |
11 = note: async blocks are not executed immediately and must either take a refere
12 help: to force the async block to take ownership of `db` (and any other reference
13 |
14 5 |         let task_a = tokio::task::spawn(async move {
15 |                                     +++++

```

我们来分析一下这个错误提示，错误提示第 1 行说，任何异步块都有可能超出当前函数的生存期，但是它里面依赖的 db 是在当前函数定义的，因此在异步块执行的时候，db 指向的对象有可能会消失，从而出错。你可能已经猜到了，错误原因跟所有权相关。原因是这个 task_a 运行的生存时间段，有可能超过 main task 生存的时间段，所以 task_a 里的 async 块中直接借用 main 函数中的局部变量 db 会有所有权相关风险。

不过这里还有一点疑问，我们不是在 main 中手动 `.await` 了吗？会等待这个子 task 的返回结果，但是 Rust 并没有分析到这一点。它可能会觉得你在 `task_a.await` 这一句之前其实是有可能将 db 给弄消失的，比如手动 `drop()` 掉这个 db，或者说调用了什么别的函数，把 db 的所有权在那里给消耗了。

它在错误信息第 12 行建议我们在 async 后加 move 修饰符，这样指明强制将 db 的所有权移动进 task_ 里去。我们按照建议修改一下。

```

1 #[tokio::main]
2 async fn main() {
3     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
4
5     let task_a = tokio::task::spawn(async move {
6         db[4] = 50;
7     });

```

```

8     _ = task_a.await.unwrap();
9
10    println!("{:?}", db);
11 }

```

仍然编译出错，报错信息换了。

 复制代码

```

1 error[E0382]: borrow of moved value: `db`
2   --> src/main.rs:10:22
3   |
4 3 |         let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
5   |         ----- move occurs because `db` has type `Vec<u32>`, which does no
6 4 |
7 5 |         let task_a = tokio::task::spawn(async move {
8   |         -----
9 6 | |             db[4] = 50;
10  | |             -- variable moved due to use in generator
11 7 | |         });
12  | |         |----- value moved here
13 ...
14 10 |         println!("{:?}", db);
15   |                               ^^ value borrowed here after move

```

它说，db 已经被移动到了 task_a 里了，所以在 main 函数中访问不到。这种严苛性，对于从其他语言过来的新手来说，确实令人崩溃。不过反过来想想 Rust 确实把细节抠得很死，这也是它比其他语言安全的原因。

那么我们就听小助手的话，不在 main 函数中打印了。这样确实能编译通过。

 复制代码

```
1 #[tokio::main]
2 async fn main() {
3     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
4
5     let task_a = tokio::task::spawn(async move {
6         db[4] = 50;
7     });
8     _ = task_a.await.unwrap();
9 }
```

第一步走通了，下一步我们要测试多任务并发的情况，所以我们需要再增加一个任务。

 复制代码

```
1 use tokio::task;
2
3 #[tokio::main]
4 async fn main() {
5     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
6
7     let task_a = task::spawn(async move {
8         db[4] = 50;
9     });
10    let task_b = task::spawn(async move {
11        db[4] = 100;
12    });
13    _ = task_a.await.unwrap();
14    _ = task_b.await.unwrap();
15 }
```

这种写法明显会有问题，我们甚至不需要编译就可以知道，因为出现了两次 `async move` 块。如果你是一步步学过来的话，应该知道，`db` 在第一个 `async move` 时已经被移动进 `task_a` 了，后面不可能再移动进 `task_b`。

编译验证后，确实如此。

 复制代码

```
1 error[E0382]: use of moved value: `db`
2     --> src/main.rs:10:30
```

```

3 |
4 5 |         let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
5 |         ----- move occurs because `db` has type `Vec<u32>`, which does no
6 6 |
7 7 |         let task_a = task::spawn(async move {
8 |         -----
9 8 |         |         db[4] = 50;
10 |         |         -- variable moved due to use in generator
11 9 |         |     });
12 |         | ----- value moved here
13 10 |         let task_b = task::spawn(async move {
14 |         -----^
15 11 |         |         db[4] = 100;
16 |         |         -- use occurs due to use in generator
17 12 |         |     });
18 |         | -----^ value used here after move

```

那应该怎么办呢?

方案三：利用 Arc

回想一下 [第 12 讲](#) 我们讲到过的 Arc 这个智能指针，它可以让多个持有者共享对同一资源的所有权。但是 Arc 也有一个巨大的限制，就是它无法修改被包裹的值。但不管怎样，我们还是碰碰运气，改动一下。

 复制代码

```

1 use std::sync::Arc;
2
3 #[tokio::main]
4 async fn main() {
5     let mut db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
6     let arc_db = Arc::new(db);
7     let arc_db2 = arc_db.clone();
8
9     let task_a = tokio::task::spawn(async move {
10         arc_db[4] = 50;
11     });
12     let task_b = tokio::task::spawn(async move {
13         arc_db2[4] = 100;
14     });
15     _ = task_a.await.unwrap();
16     _ = task_b.await.unwrap();
17
18     // println!("{:?}", db);

```

```
19 }
```

不出所料，Rust 编译不通过，这说明通过 `Arc<T>` 没办法修改里面的值。

[复制代码](#)

```
1 error[E0596]: cannot borrow data in an `Arc` as mutable
2   --> src/main.rs:10:9
3   |
4 10 |         arc_db[4] = 50;
5   |         ^^^^^^^ cannot borrow as mutable
6   |
7   = help: trait `DerefMut` is required to modify through a dereference, but it i
8
9 error[E0596]: cannot borrow data in an `Arc` as mutable
10  --> src/main.rs:13:9
11  |
12 13 |         arc_db2[4] = 100;
13  |         ^^^^^^^^ cannot borrow as mutable
14  |
15  = help: trait `DerefMut` is required to modify through a dereference, but it i
```

虽然现在我们的代码还是没有编译通过，但是思路是没问题的：要在并发的多个任务中，访问同一个资源，那么必然涉及到多所有权，所以使用 `Arc` 是完全没有问题的。现在的问题是，**有没有办法更改被 `Arc` 包裹起来的值。**

答案是有的，利用 `Mutex` 就可以。

方案四：Arc+Mutex

一个多任务并发编程要修改同一个值，那必然要防止修改冲突，这就不得不用到计算机领域里一个常见的工具——锁。

`Mutex` 是一种互斥锁，被 `Mutex` 包裹住的对象，同时只能存在一个 reader 或一个 writer。使用的时候，要先获得 `Mutex` 锁，成功后，才能读或写这个锁里面的值。多个任务不能同时获得同一个 `Mutex` 锁，当一个任务持有 `Mutex` 锁时，其他任务会处于等待状态，直到那个任务用完了 `Mutex` 锁，并自动释放了它。

```
1 use std::sync::Arc;
2 use tokio::sync::Mutex;
3
4 #[tokio::main]
5 async fn main() {
6     let db: Vec<u32> = vec![1,2,3,4,5,6,7,8,9,10];
7     let arc_db = Arc::new(Mutex::new(db)); // 加锁
8     let arc_db2 = arc_db.clone();
9     let arc_db3 = arc_db.clone();
10
11     let task_a = tokio::task::spawn(async move {
12         let mut db = arc_db.lock().await; // 获取锁
13         db[4] = 50;
14         assert_eq!(db[4], 50); // 校验值
15     });
16     let task_b = tokio::task::spawn(async move {
17         let mut db = arc_db2.lock().await; // 获取锁
18         db[4] = 100;
19         assert_eq!(db[4], 100); // 校验值
20     });
21     _ = task_a.await.unwrap();
22     _ = task_b.await.unwrap();
23
24     println!("{:?}", arc_db3.lock().await); // 获取锁
25 }
26 // 输出
27 [1, 2, 3, 4, 100, 6, 7, 8, 9, 10]
```

加上 Mutex，这个例子就能顺利编译并运行通过了。

这个例子中的第 7 行，我们使用 `Arc::new(Mutex::new())` 组合把 db 包了两层，外层是 Arc，里层是 Mutex。然后，我们把 arc_db 克隆了两次，这种克隆只是增加 Arc 的引用计数，代价非常低。

然后在每次使用的时候，先通过 `arc_db.lock().await` 这种方式获得锁，再等待取出锁中对象的引用，这里也就是 Vec 的引用，然后通过这个引用去更新 db 的值。

利用 Arc 与 Mutex 的组合，我们还顺便解决了在 main task 不能打印这个 db 的问题。实际上，在 Rust 中，`Arc<Mutex<T>>` 是一对很常见的组合，利用它们的组合技术，基本上可以

满足绝大部分的并发编程场景。

有了这两兄弟的加持，我们用 Rust 写业务代码就变得像 Java 一样高效、便捷。相对于 Go、Python 或 JavaScript 来说，Rust 的异步并发编程代码稍微有些繁琐，但是它的模式是非常固定的，最后这个示例里的模式可以无脑使用。正因为如此，Arc、Mutex 和 clone() 一起，被社区叫做“Rust 三板斧”，就是因为它们简单粗暴，方便好用。

到这里为止，我们已经解决了这节课开头提出的问题。

其他锁

除了 Mutex，tokio 里还提供了一些锁，我们来看一看。

tokio::sync::RwLock

RwLock 是读写锁。它和 Mutex 的区别是，Mutex 不论是读还是写，同时只有一个能拿到锁。比如，一个 task 在读，而另一个 task 也想读的时候，仍然需要等待第一个 task 先释放锁。所以在读比较多的情况下，Mutex 的运行效率不是太理想。

而 RwLock 的设计是，当一个任务拿的是读锁时，其他任务也能再拿到读锁，多个读锁之间可以同时存在。当一个任务想拿写锁的时候，必须等待其他所有读锁或写锁释放后才能拿到。

当一个任务拿到了写锁时，其他任务只能等待它完成后才能继续操作，不管其他任务是要写还是读。因此对于写来讲，RwLock 是排斥型访问；对于读来讲，RwLock 提供了共享访问。这一点与不可变引用和可变引用的关系特别像。

我们来看下面的示例：

 复制代码

```
1 use tokio::sync::RwLock;
2 #[tokio::main]
3 async fn main() {
4     let lock = RwLock::new(5);
5     // 多个读锁可以同时存在
6     {
```

```

7     let r1 = lock.read().await;
8     let r2 = lock.read().await;
9     assert_eq!(*r1, 5);
10    assert_eq!(*r2, 5);
11 } // 在这一句结束时，两个读锁都释放掉了
12
13 // 同时只能存在一个写锁
14 {
15     let mut w = lock.write().await;
16     *w += 1;
17     assert_eq!(*w, 6);
18 } // 在这一句结束时，写锁释放掉了
19 }
```

可以看到，RwLock 的使用非常简单，在读操作比写操作多很多的情况下，RwLock 的性能会比 Mutex 好很多。

Rust 标准库中还有一些用于简单类型的原子锁。

std::sync::atomic

如果共享数据的只是一些简单的类型，比如 bool、i32、u8、usize 等等，就不需要使用 Mutex 或 RwLock 把这些类型包起来，比如像这样 `Arc<Mutex<u32>>`，可以直接用 Rust 标准库里提供的原子类型。`std::sync::atomic` 这个模块下面提供了很多原子类型，比如 AtomicBool、AtomicI8、AtomicI16 等等。

`Mutex<u32>` 对应的原子类型是 `std::sync::atomic::AtomicU32`。

像下面这样使用：

 复制代码

```

1 use std::sync::atomic::AtomicU32;
2
3 fn main() {
4     // 创建
5     let atomic_forty_two = AtomicU32::new(42);
6     let arc_data = Arc::new(atomic_forty_two);
7
8     let mut some_var = AtomicU32::new(10);
9     // 更新
```

```
10     *some_var.get_mut() = 5;
11     assert_eq!(*some_var.get_mut(), 5);
12 }
```

其他类型按类似的方式使用就可以了。原子类型之所以会单独提出来，是因为它是锁的基础，其他的锁会建立在这些基础原子类型之上，这些原子类型也可以充分利用硬件提供的关于原子操作的支持，从而提高应用的性能。

注：在多核 CPU 中，常见的硬件支持原子操作的方法包括 CPU 中的缓存一致性协议、总线锁定等机制，可以使多个线程或进程同时对同一变量进行原子操作时，不会出现数据竞争和线程同步的问题。

锁 (lock) 和无锁 (lock-free) 是计算机科学领域一个非常大的课题，Rust 有本书 [🔗 《Rust Atomics and Locks》](#) 专门讲这个，有兴趣的话你可以看一看。

小结

这节课我们通过一步步验证的方式，学习了在 Rust 和 tokio 中如何在多个任务中操作共享数据，经过多次被编译器拒绝的痛苦，最后我们得到了一个相当舒服的方案。这个方案可以供我们以后在做并发编程时使用，使用时的模式非常固定，没有什么心智负担。整个探索过程虽然比较辛苦，但是结果却是比较美好的。这也许就是 Rust 的学习过程吧，先苦后甜。

在整个探索过程中，我们也能深切体会 Rust 所有权模型在并发场景下发挥的重要作用。如果你想让程序编译通过，那么必须严格遵守 Rust 的所有权模型；一旦你在 Rust 的所有权指导下捣鼓出了并发代码，那么**你的并发代码就一定不会产生由于竞争条件而导致的概率性 Bug**。

如果你有这方面的经验教训，你一定会特别憎恶这种概率性的 Bug，因为有可能仅仅是重现 Bug 现场就要花一个月的时间。同时，你会爱上 Rust，因为它从语言层面就帮我们杜绝了这种情况，让我们的线上服务特别稳定，晚上可以安心睡觉了。

思考题

这节课代码里下面这两句的意义是什么，第一行会阻塞第二句吗？

复制代码

```
1    _ = task_a.await().unwrap();  
2    _ = task_b.await().unwrap();
```

希望你可以开动脑筋，认真思考，把你的答案分享到评论区，也欢迎你把这节课的内容分享给其他朋友，邀他们一起学习 Rust。好了，我们下节课再见吧！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (15)

最新 精选



lilp

2023-11-23 来自广东

1. 应该是防止这两个任务还没走完，主线程就结束了？
2. 不会阻塞。我理解的是这两个任务 同时启动，顺序完成。不管他俩怎么去抢这个锁 最后的完成顺序应该还是和main中写的.await() 顺序一样。

作者回复: 1 是的。 2 spawn后就立即执行。后面这两句只是等待任务返回。在执行第二句之前有可能第二个任务已经执行完了，第二句只是把其任务的返回值取回来。

共 2 条评论 >

👍 3



-

2023-11-22 来自北京

有个疑问，Arc::new(Mutex::new(db))后可以将一个不可变的变量变成可变变量？这个是什么原因

作者回复: 思考得很棒，前后联系起来了。arc mutex 其实是拿到了对象所有权，有所有权了当然修改值就方便了。跟前面并不冲突，当然如果还想深入下去，可以看看 内部可变性 这个概念。

共 2 条评论 >

👍 2



下雨天

2023-11-23 来自湖北

看实现，就当前例子而言task_a不会阻塞task_b。 如果task_a中loop{}下就可以阻塞了。

有个疑问，为啥task::spawn后面会自动执行呢？ 我理解只有.await了才会加到调度器里面执行。

作者回复: spawn比较特殊，特殊处理的

共 2 条评论 >

👍 1



PEtFiSh

2023-11-23 来自上海

await代码会持续等待直到任务结束，因此在main thread里第一行会阻塞第二行。但这不会让task_a阻塞task_b。加入await可以使最后的println!打印两个任务执行完以后被修改的db值，如果不加入await。有一定几率最后println!打印的还是原始的db

作者回复: 👍



👍 1



Geek_72807e

2023-11-22 来自山西

请问老师，方案四中，会不会出现两次修改操作顺序不确定的问题，最终结构可以是40，也会是100？

作者回复: 很棒的思考 👍，有可能出现。看哪个任务后执行完成，后抢到锁。

共 2 条评论 >

👍 1



superggn

2023-12-21 来自北京

问个事儿， Arc::clone(arc_db) 和 arc_db.clone() 一样吗？

为啥 不用 `std::sync::{Arc, Mutex}` 而是 `tokio::sync::{Arc, Mutex}`

如果是因为在 tokio runtime 里头要用 tokio 的东西的话, 那么在啥情况下会用 std 的几板斧呢?

同步场景都用 std::sync 底下的东西吗?

作者回复: 1. Arc::clone(arc_db) 和 arc_db.clone() 是一样的。

2. 教程里面用 tokio 的 Arc, Mutex, 主要是不给同学引入额外的理解负担。不然要学的东西又多了。

关于它们的差别, 可以看这里: <https://tokio.rs/tokio/tutorial/shared-state> On using std::sync::Mutex

共 3 条评论 >



superggn

2023-12-21 来自北京

思考题

意义: 等执行完了再打印, 对齐一下子, 别任务没跑完主进程就完事儿了

到 await 这块儿就是同步了, 这俩 await 是顺序执行,

如果 task_a 没执行完, task_b 已经执行完了, 就会卡在 task_a 这里, task_a 这行完事儿了在执行 task_b 的 await, 这会儿 task_b 因为已经完事儿了, 所以第2个 await 不会卡

作者回复: 很棒



-Hedon

2023-12-20 来自湖北

task_a.await.unwrap() 是阻塞等待任务结果, 所以 task_a.await.unwrap() 会阻塞 task_b.await.unwrap(), 但是 task_a 不会阻塞 task_b, spawn 并发执行的。

作者回复: 对的。



Geek_e5eb33

2023-12-15 来自安徽

请问老师，rust 中不建议使用全局变量。那如果我想进行模块化开发，在 A 模块中定义的变量(比如缓存了用户信息)，怎么供其他模块使用呢，目前能想到的是都定义到 main 里。

作者回复：通过函数参数传递进来。生成的所有权对象通过函数值返回。



Taozi

2023-11-22 来自上海

关于arc里面为何需要套lock才能修改值，当然是为了在运行时保证内存安全。为什么说在运行时，因为还有对应的在静态时。还记得最开始我们说rust中每个值都有一个owner，这是为了保证ower在其作用域结束时释放值，这是可以通过代码静态分析出来的。对于所有权不能静态确定的情况，就需要arc来这个第三方来持有所有权，然后动态的决定何时释放值。原理也很简单，就是引用计数。同样最开始我们也说一个值只能同时存在一个可变借用或者多个不可变借用，这也是可以通过静态分析保证的，但是在arc这里只能通过加锁在运行时保证这点。

作者回复：👍 很棒的分析，赞

